An Introduction to Computation



Computer Science has a long history. History is empowering and fun. We build on our past. So let's learn where computing comes from, why we do it, and what we can to with it.

CONTENTS

What is Computation? • A Brief History • Other Histories • Beyond History • Big Ideas • Theories and Practices • Recall Practice • Summary

What is Computation?

Information is that which informs.

Computation is the *mechanical* generation of new information from old information.

Computations are expressed within a **programming language** and carried out with **automata**.

The study of computation as a discipline is therefore the study of languages and automata.

The human understanding of, and application of, computing has evolved over millennia. It has arguably enhanced human capabilities. There is a history here that is empowering to know. History is an essential component of the study of any discipline. However:

> *[Computing is] complete pop culture. The big problem ... [is that] the electronic media we have is so much better suited for transmitting pop-culture content than it is for high-culture content. I consider*

jazz to be a developed part of high culture. [Jazz aficionados take deep pleasure in knowing the history of jazz, but] pop culture holds a disdain for history. Pop culture is all about identity and feeling like you're participating. It has nothing to do with cooperation, the past, or the future—it's living in the present. I think the same is true of most people who write code for money. They have no idea where [their culture came from]. —Alan Kay

Woah. He's right. Let's do something about this.

A Brief History

Here are some important events, people, works, and machines to be aware of in order to have a proper context for the study of computation.

In ancient times

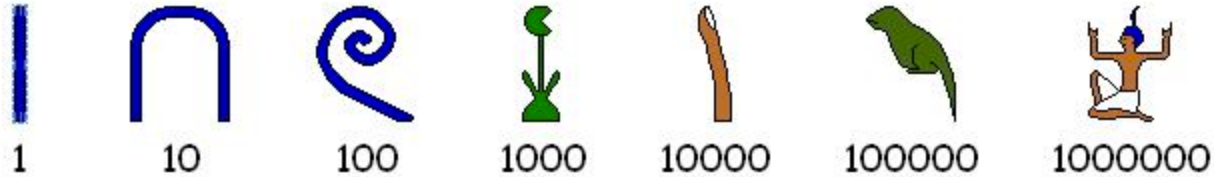
Since antiquity, humans have been marking, matching, comparing, tallying, measuring, and reckoning.



A big step

Early inventories featured pictorial markings like 🌽🌽🌽🌽🌽🍓🍓🍓. Then someone figured out they could save space by writing something like 5🌽 3🍓. An incredibly powerful idea: symbols representing quantities!

Numerals!



Early numeral systems like the Egyptian system were additive, but today positional systems are universal.

> ***Exercise**: Browse Wikipedia's List of Numeral Systems.*

Recipes for calculation

Recipes, or lists of instructions, for manipulating quantities and measurements were sometimes **recorded**. Notable examples include:

- Babylonian Tablets 1800-1600 BC
- Rhind Mathematical Papyrus (~1550 BC)
- Euclid's greatest common divisor method (~300 BC)
- Eratosthenes method for enumerating primes (~240 BC)
- Muḥammad ibn Mūsā al-Khwārizmī's Al-Jabr (813-833), which included systematic solutions for linear and quadratic equations





Calculating machines

While humans could follow recipes and carry out reckoning with their fingers and toes, or use tally marks on bones, gains in efficiency and accuracy naturally arise with the creation of mechanical devices for calculation. Here are just a few examples:

- Abacus (earliest known from Sumeria 2700–2300 BC)
- Astrolabe (~200 BC)
- Antikythera mechanism (~200 BC)
- Slide Rule (1620s)
- Pascal's calculators (1640s)

- Jacquard Loom (1804)
- Babbage's Difference Engine (1820s) and Analytical Engine (1837)
- Hollerith's Tabulating Machine (1880s)






The Analytical Engine was fascinating historically. Although never built, it was perhaps the world's first **programmable**, **general-purpose** computer. Several programs are known to have been written for the device, the most famous of which—a program for the computation of Bernoulli numbers—was presented by Babbage's collaborator Ada Lovelace in her translator's notes to Luigi Menabrea's article in *Taylor's Scientific Memoirs* in 1843. (Also see this shorter summary). Lovelace is often considered the world's first programmer. She is also widely admired as being perhaps the first to recognize the wide application of computing beyond arithmetic, writing that the Analytic Engine

> *might act upon other things besides number, were objects found whose mutual fundamental relations could be expressed by those of the abstract science of operations, and which should be also susceptible of adaptations to the action of the operating notation and mechanism of the engine.... Supposing, for instance, that the fundamental relations of pitched sounds in the science of harmony and of musical composition were susceptible of such expression and adaptations, the engine might compose elaborate and scientific pieces of music of any degree of complexity or extent.*

Philosophers, mathematicians, and logicians take interest

The 19th century saw wild advances in formal logic and the foundations of mathematics. Logic broke free of Aristotlean syllogisms. Cantor introduced us to the paradise of multiple infinities. Three philosophies about math even was emerged:

Logicism	Intuitionism	Formalism

Math is just logic

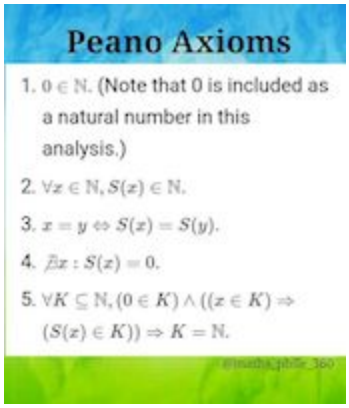
Math is only what we invent and can construct or demonstrate

Math is done by manipulating symbols according to rules

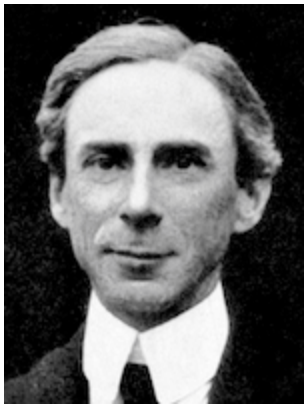
Major works of this era include:

- Boole's Boolean Algebra (1847)
- Frege's Begriffsschrift (1879)
- Peano's Axioms (1889)
- Hilbert's Program (early 1920s)
- Russell and Whitehead's *Principia Mathematica* (1910–1913)









People learned that formalizing mathematics, or even giving it a logic foundation, was not so direct. Any attempt at providing a consistent, formal, axiomatic basis for mathematics (that included arithmetic) could never capture all truths. A shocking result at the time! Gödel proved this in 1931.

Computation gets formalized

Though Gödel showed there could be no logically consistent and complete formalization of all of mathematics, the question of whether there existed an effective, finite, **decision procedure** for all statements of mathematics remained open. (Yay, we're back to computation!) This question is known as the Entscheidungsproblem. Hilbert thought such a procedure existed. Others did not. To prove him wrong, one needed to formalize the very idea of algorithm. The three main approaches of the 1930s were:

- Church's Lambda Calculus

- The μ-recursive Functions (mostly worked out by Gödel, Kleene, and others)
- Turing Machines





Church and Turing both showed the Entscheidungsproblem had no solution. Poor Hilbert.

> ***Story Time***
>
> *We'll go over the story of Church, Gödel, and Turing in class only. You can't find everything on the notes!*

Turing

Turing's paper is one of the most impactful and significant of all time.

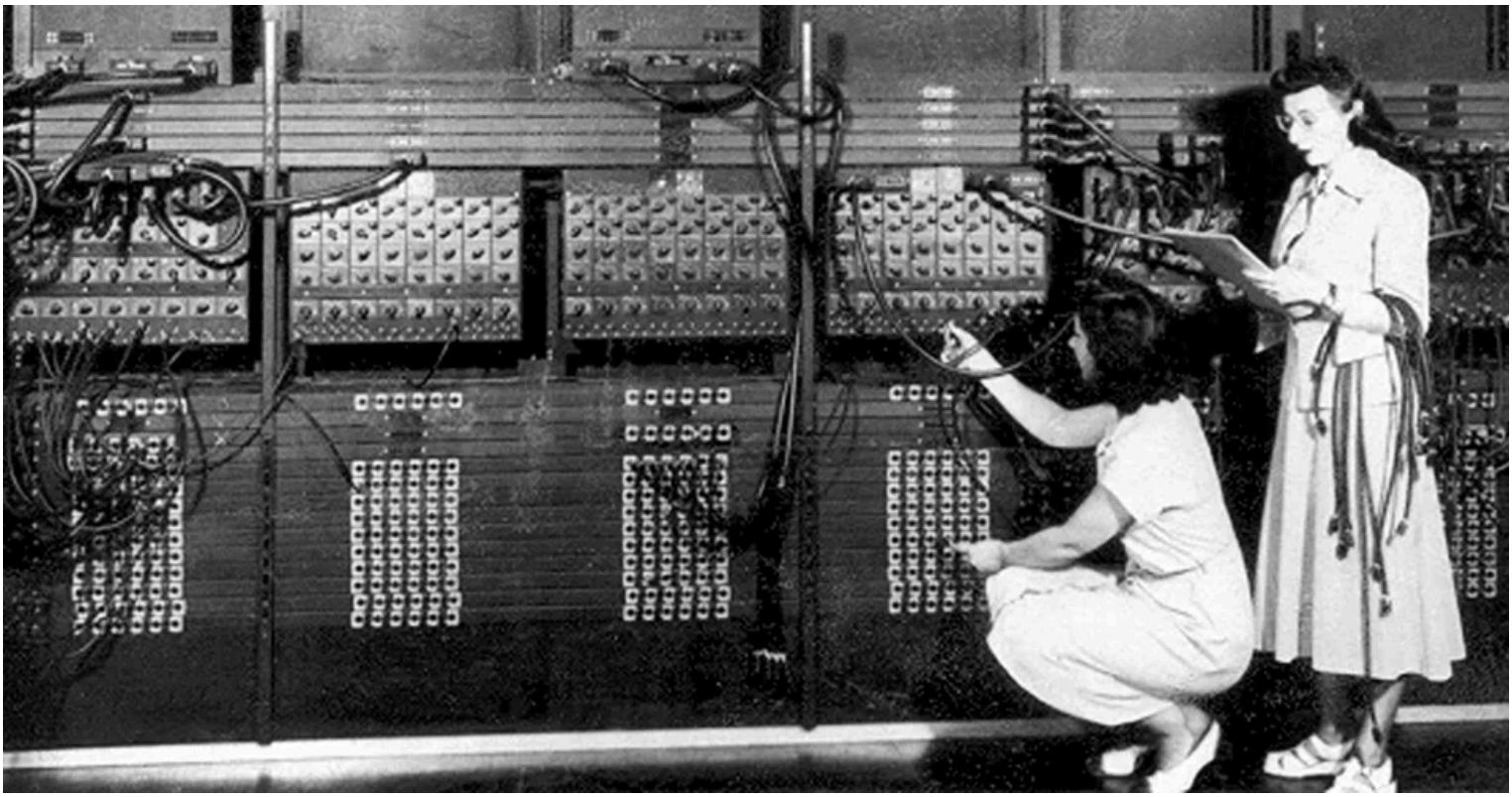
- He gave a **formalization of effective computation** (algorithms) in a way people could all understand
- He gave evidence as to why his formalization, now called the Turing Machine, **coincides with our intuitive notion of "computable"** (the Church-Turing Thesis).
- He introduced **computational universality**: programs were just data (ok so far, not surprising) and, wait for it, a single machine could emulate, or "run," any other machine (!!!!!!!!!!!!!!!!). Before this work, it was not known (but it may, we don't know, have been suspected by some) that a truly universal computer could exist—separate machines were thought to be necessary for different tasks.
- He showed that some numbers were not computable, some functions were not computable, and that some questions could not be mechanically decided—and hence, that the Entscheidungsproblem had no solution.

Details coming soon

We'll study Turing's paper, Turing machines, and the notion of universal computation in detail later in this course. Why? Turing is revered as the founder of computer science because of this work. The ideas that there are limits to computation, equivalent ways to express computation, and the existence of a truly universal computing machine make computer science a discipline.

Electronic computing machines

Turing's Universal Machine (1936) opened up a flood of new work in general purpose machines. Wikipedia's History of Computing Hardware page covers both ancient devices and a fair number of machines from the 1930s–1960s. It is worth a look to see how we eventually got to the electronic devices that are ubiquitous today.



Programming Languages

von Neumann once angrily asked someone writing an assembler "Why would you ever want more than machine language?" Today that question sounds silly. Even assembly language is too low-level.

One of the biggest proponents of high-level languages was Grace Hopper, who also happened to write the first compiler. Here are some notable high-level languages:

- Fortran, COBOL, LISP
- Algol, PL/1, Simula
- APL, J, K, Q
- CLU, Alphard, Euclid, Mesa, Cedar, Turing, Ada
- BCPL, B, C, C++, D, Rust, Zig, Odin, Swift
- Java, C#, Scala, Kotlin
- Perl, PHP, Hack, Tcl, Python, Ruby, Groovy, Lua
- Simula, Smalltalk, Object Pascal, Eiffel
- Concurrent Pascal, Occam, Linda, SR, Erlang, Elixir, Parasail, Chapel, Go
- Self, JavaScript, ActionScript, CoffeeScript, TypeScript
- Standard ML, CAML, Ocaml, F#, Elm, Reason
- Miranda, Haskell, PureScript, Idris
- Prolog, Mercury, Oz, Curry
- Agda, Coq, Lean
- Snobol, Icon
- Id, Val, Sisal
- SQL, Cypher, SPARQL
- Postscript
- MATLAB, R, Julia, Mojo
- Intercal, Unlambda, Brainfuck, Malbolge, Befunge, ZT, Java2K, LOLCODE
- GolfScript, Pyth, CJam, Jelly, 05AB1E, Stax



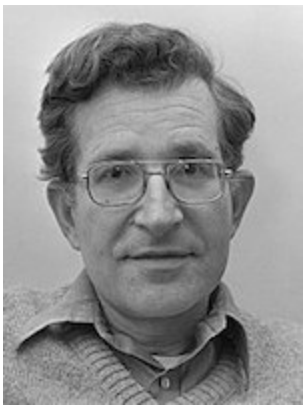



***Exercise**: The languages above are informally grouped. Try to determine, for each line, which characteristic is shared by the languages on that line. You'll likely need to do a lot of research, so don't spend too much time on this. Just focus on the lines that may interest you the most.*

Language Implementation

Languages have to not only be expressive, but they have to be efficiently implementable. This is why we study compilers and interpreters. Two major parts of this field are:

- Linguistic Theories (from Chomsky and others) that enable parsing to be efficient
- Optimizations (Frances Allen was a major figure in this area)




Human-centric computing

People create programs for people. Areas with people-focus include:

- Graphics and user interfaces
- Computing in education
- Libraries for creative computing



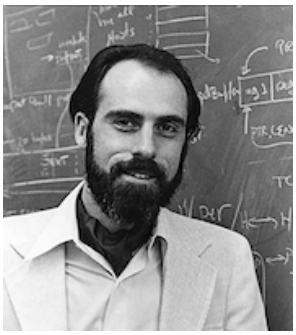



***Exercise**: Read about Douglas Engelbart's Mother of All Demos (1968).*

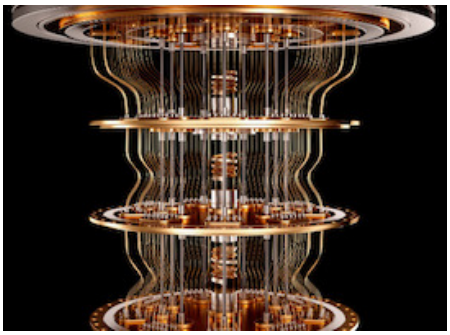
Modern Trends

Trends in the modern computing era:

- Personal computers (1970s)
- Internet (1980s)
- Web (1990s)
- Mobile (2000s)
- Cloud (2010s)
- Generative AI (2020s)
- Quantum Computing (?)



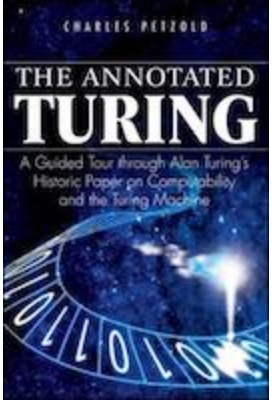




Other Histories

Computing history is so much richer than the brief outline above. The following are highly recommended:

- Gabriel Robins' remarkable A Brief History of Computing
- Wikipedia's Computing History article
- Wikipedia's Computing Timeline article
- Charles Petzold's fabulous book The Annotated Turing



Beyond History

Computing is so much more than its technical core, its theoretical concepts, or even its history. Computing is a human activity. Computations, programming languages, and machines, are designed by humans for humans.

Grady Booch is building a remarkable [transmedia documentary on the intersection of computing and the human experience](#).



The web version introduces itself as:

> *The story of computing is the story of humanity: this is a story of ambition, invention, creativity, vision, avarice, power, and serendipity, powered by a refusal to accept the limits of our bodies and our minds. Computing: The Human Experience is a transmedia project that explores the science of computing, examines the connections among computing, individuals, society, and nations, and by considering the history of computing contemplates the forces that will shape its future.*

“powered by a refusal to accept the limits of our bodies and our minds”

It is worth immersing yourself in this content for hours.

> ***Exercise**: Spend an hour exploring the site.*

Big Ideas

During humans’ creation of a science, art, and craft of computing, several major themes have emerged. They form the core of the field that we now call **computer science**. Among these themes, or big ideas, are:

- ✅ Symbols and symbol manipulation **amplify** our thought processes.
- ✅ The functions that can be computed comprise an **infinitesimal fraction** of the functions we know that exist.
- ✅ Theoretical computing devices (such as Turing Machines) are important for showing the **limits of computation**, but are not suitable for real computation.
- ✅ **Universal computation** is far, far from obvious, but we take it for granted today.
- ✅ Modern computers speed up time to such an extent that what we know and what we can know is **qualitatively different** than what we could know before computing.
- ✅ **Programming languages** enable the expression of computation at such a high level that creative and impactful computing become accessible to the masses.

Theories and Practices

In studying any discipline, humans create **theories**. A theory is an organized body of knowlege with explanatory and predictive powers. There are four major theories of computation:

- **Language Theory**, dealing with how computations are expressed
- **Automata Theory**, dealing with how computations are carried out

- **Computability Theory**, dealing with what is and is not computable
- **Complexity Theory**, dealing with how hard (in terms of resources like time and space) computations are

The practice of computing involves gaining proficiency in crafting computations (programs) in dozens of different programming languages and in many different programming styles, applying concepts, patterns, and idioms. It also includes the design and implementation of new programming languages and even new machines.

Designing one's own language and implementing compilers and interpreters are essential activities in building a deeper understanding of computation.

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What is computation? HIDE ANSWER

The formal, mechanical, generation of new information from old information.

What is a programming language? HIDE ANSWER

A language for expressing computations.

What is an automaton? HIDE ANSWER

A machine for carrying out computations.

According to Alan Kay, why is computing history not widely known? HIDE ANSWER

Because pop culture, which computing arguably is, holds a disdain for history.

What is the significance of numerals? HIDE ANSWER

They allow large quantities to be expressed with fewer symbols.

Show how to represent the quantity twenty-one thousand two hundred thirty-seven using Egyptian Numerals: HIDE ANSWER



What are some ancient media on which early recipes for calculation were recorded?
HIDE ANSWER

Babylonian Tablets and the Rhind Mathematical Papyrus.

What are some early famous nontrivial algorithms? HIDE ANSWER

Euclid's greatest common divisor method and Eratosthenes method for enumerating primes

What were some of the accomplishments of Al-Khwārizmī? HIDE ANSWER

He wrote *Al-Jabr*, which included systematic solutions for linear and quadratic equations.

Where and when were abaci first known to be used? HIDE ANSWER

Sumeria 2700–2300 BC.

What could the Antikythera mechanism do? HIDE ANSWER

It was an analog computer that could predict astronomical positions and eclipses.

Babbage is known for designing to machines. Which ones? HIDE ANSWER

The Difference Engine and the Analytical Engine.

Was the analytical engine ever built? HIDE ANSWER

No.

What was Ada Lovelace's published program in her famous *Notes*? HIDE ANSWER

A program for the computation of Bernoulli numbers.

What was Lovelace most known for besides the first published program? HIDE ANSWER

She was arguably the first to recognize and to write about the wide application of computing beyond arithmetic.

What did George Boole introduce to the world? HIDE ANSWER

Boolean Algebra.

What was Hilbert's Program? HIDE ANSWER

A call to people of his time to formalize all of mathematics.

What did the logicists think math was? What did the formalists think it was? The intuitionists? HIDE ANSWER

Logicists: a branch of logic. Formalists: a game of manipulating symbols according to rules. Intuitionists: an invention for constructing objects and facts about them.

What was the Entscheidungsproblem? HIDE ANSWER

Hilbert's question of whether there was an effective, finite, decision procedure for all statements of mathematics.

How did Church formalize the notion of effective computability? HIDE ANSWER

With the Lambda Calculus.

How did Turing formalize the notion of effective computability? HIDE ANSWER

With the Turing Machine.

What was Gödel's initial reaction to Church's Lambda Calculus, and what did this reaction motivate Gödel to do? HIDE ANSWER

He was skeptical that the Lambda Calculus was a model for effective computability, so he created his own mode, the μ-recursive functions.

After Church demonstrated the equivalence of the Lambda Calculus and the μ-recursive functions, what was Gödel's reaction? HIDE ANSWER

He thought his own model, the μ-recursive functions, must be insufficient.

When did Gödel finally accept the Lambda Calculus a model for effective computability? HIDE ANSWER

After Turing showed that the Lambda Calculus and Turing Machines were equivalent. All agreed that Turing machines were a sufficient model, thus the other two approaches must also be.

Name four stunning achievements of Turing's famous paper. HIDE ANSWER

He gave a formalization of effective computation, showed that his formalization coincided with our intuitive notion of "computable", introduced computational universality, and showed that some numbers (and by extension some functions) were not computable.

Why was the ENIAC famous? HIDE ANSWER

It was the first general-purpose electronic computer.

Who wrote the first compiler and what was it called? HIDE ANSWER

Grace Hopper. The A-0 System.

Why are programming languages important? HIDE ANSWER

They enable the expression of computation at such a high level that creative and impactful computing become accessible to the masses.

Why is LISP so loved? HIDE ANSWER

Simple syntax and semantics, homoiconicity, macros.

Why is Ruby so loved? HIDE ANSWER

It is extremely expressive, allows for rapid development, and has excellent metaprogramming facilities.

Why is CLU so significant? HIDE ANSWER

It introduced data abstraction, iterators, and exception handling.

What else is Barbara Liskov known for? HIDE ANSWER

She introduced the Liskov Substitution Principle.

What was Noam Chomsky's biggest contribution to computer science? HIDE ANSWER

He introduced the Chomsky Hierarchy, which served as a foundation for modern parsing theory.

What is Frances Allen known for? HIDE ANSWER

She was a (if not the) major figure in the field of compiler optimization. It is said that among all the optimizations performed in modern compilers, she probably came up with or at least described 80% or more of the most important ones.

What year did Allen win the Turing Award? HIDE ANSWER

2006.

What even is Generative AI? HIDE ANSWER

AI that can create new things, like images, music, code, or text.

Where can I find more histories of computing? HIDE ANSWER

Gabriel Robins' A Brief History of Computing, Wikipedia's Computing History article, Wikipedia's Computing Timeline article, and Charles Petzold's The Annotated Turing.

What can I find at *Computing: the Human Experience*? (You should visit and explore the site to answer) HIDE ANSWER

Histories, timelines, stories, articles, films, interactive exhibits, and educational resources on computing and its impact on humanity (considering individuals, cultures, nations, and societies).

What are the four major theories of computation? HIDE ANSWER

Language Theory, Automata Theory, Computability Theory, and Complexity Theory.

Learning about computing requires a studying languages and automata. Why do we study these two subjects? HIDE ANSWER

Languages are for expressing computations, and automata are for carrying out computations.

Summary

We've covered:

- ✅ Earliest notions of computing
- ✅ Numbers as abstractions of quantities
- ✅ Numerals as representations of numbers
- ✅ Earliest known recordings of recipes for calculations
- ✅ Early computing machines
- ✅ Influence of the analytical engine
- ✅ Ada Lovelace's insights
- ✅ Math in the late 1800's
- ✅ The formalization of computing
- ✅ Undecidability
- ✅ Human-centric computing
- ✅ The modern era in computing
- ✅ Several big ideas of computing
- ✅ Computing Theories

Theories in Computer Science

To develop expertise in any endeavor, you need both practice and theory. Computer Science is a discipline with many practices and many theories. Let's visit a few of the major theories—the science in Computer Science. Why? Don't worry, we'll explain the usefulness of studying theory, too.

CONTENTS

What is a Theory? • Why Study Theory? • Theories of Computation • Language Theory • Automata Theory • Computability Theory • Complexity Theory • Working With These Theories • Theories and Creativity, Again • Recall Practice • Summary

What is a Theory?

We'll begin with this definition from an actual respected dictionary:

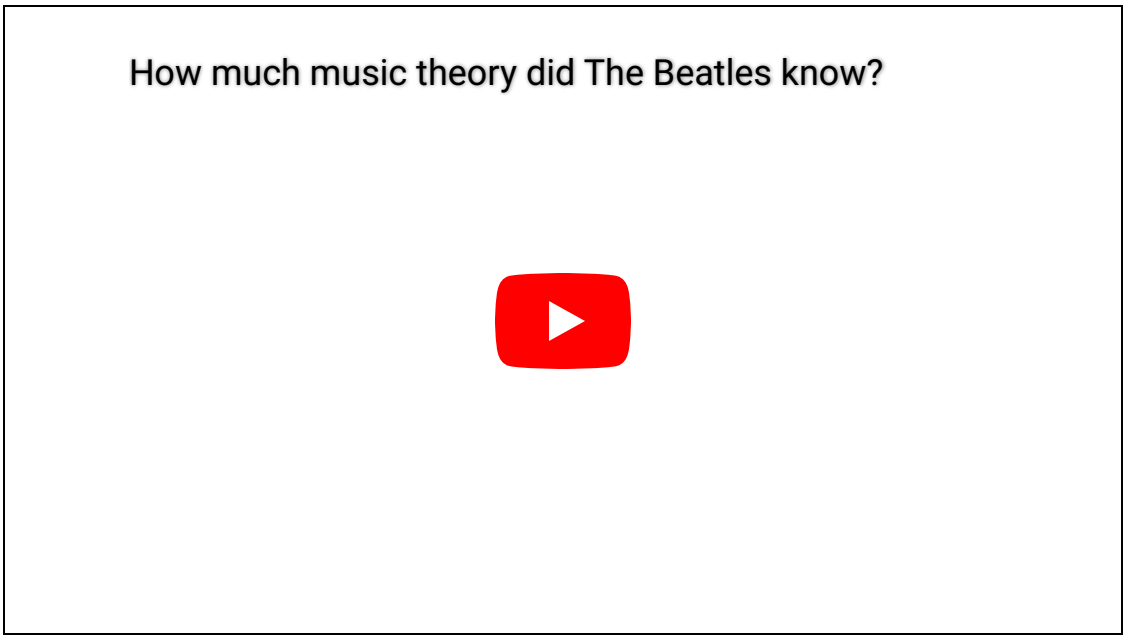
> ***theory (n.)***
>
> 1. *A set of statements or principles devised to explain a group of facts or phenomena, especially one that has been repeatedly tested or is widely accepted and can be used to make predictions about natural phenomena.*
>
> 2. *The branch of a science or art consisting of its explanatory statements, accepted principles, and methods of analysis.*

Sure, sometimes folks use the word "theory" for a hunch or a supposition, but in the arts and sciences, a theory is *an organized body of knowledge with explanatory and predictive powers*.

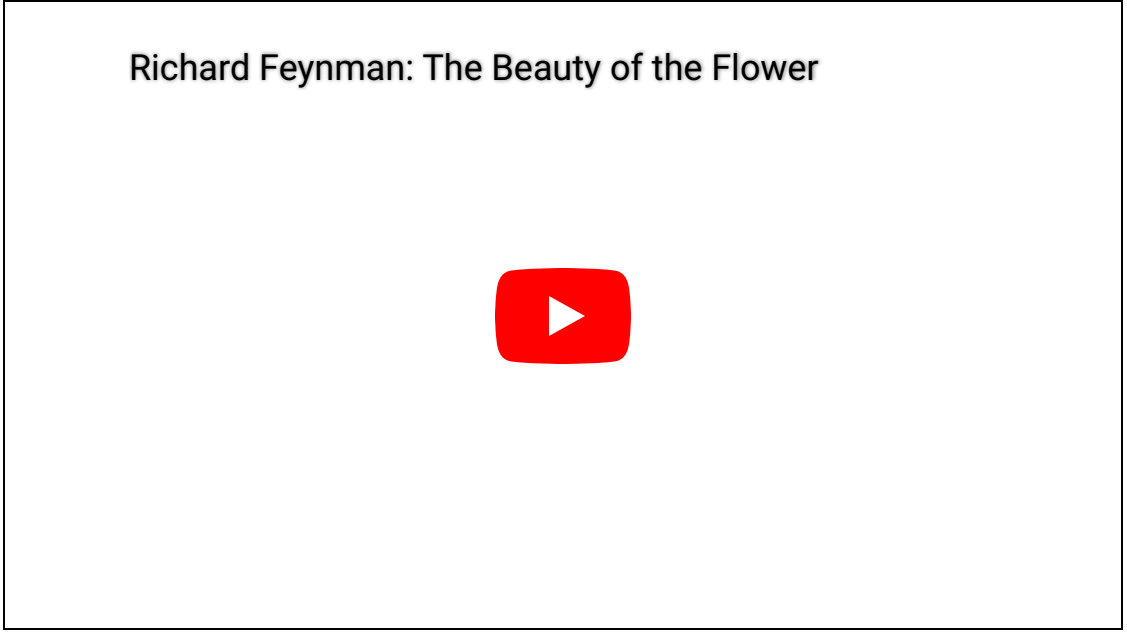
Why Study Theory?

We build a theory to **organize** our knowledge and develop a **vocabulary** that we can use to reason about things, communicate with others, make predictions, and generate new knowledge.

Without a theory, we can still do things, but not as well: there's a lot of improvisation, difficulty in communicating ideas, guess work, and flailing around. Your practice will not be as strong, and your insights may be limited. Let's see how David Bennett expertly makes this point. The key ideas are expressed from 17:21-18:42 (though the whole video is interesting):



Music theory can be helpful in creating music, but can it help us enjoy music more? Certainly! After all, theoretical knowledge *adds* to our awe of that which we study, allowing us to appreciate the beauty of something at more than just one level:



Theories of Computation

The field we now call **Computer Science** has a long and fascinating history. Several theories emerged as the discipline took shape. A **computer scientist** has a working knowledge of these theories and employs them to not only create useful applications and systems, but to advance the state of the art. There are four major theories under the umbrella term "The Theory of Computation." Two are rather fundamental:

Language Theory

How are computations **expressed**?

Automata Theory

How are computations **carried out**?

The third and fourth get into certain specifics but have deep and fascinating philosophical roots:

Computability Theory

Complexity Theory

What are the fundamental **limits** of computation?

What **resources** are required to perform certain computations?

To study and apply these theories, we need some context.

Computation augments and amplifies human thought. Modern computers speed up time to such an extent that our reasoning powers are at a qualitatively different level than what we had before these machines. These computational theories help us qualify and quantify how exactly this augmentation occurs. They enable us to ask and answer questions such as

- What even *is* computation?
- What can and cannot be known through computation?
- What can and cannot even *be* computed?
- What can and cannot be computed under certain time and space constraints?"

In order to be accepted and useful, these theories must be grounded with practical axioms that match our intuition of what computations actually are. We will take as given that a computer is a finite device and a program is a finite set of instructions.

***Exercise**: Why are the assumptions of finiteness not only reasonable but essential?* SHOW ANSWER

Language Theory

Language Theory is concerned with how information and computation are expressed. It begins with the question "What is information?" and approaches the question with the hypothesis that any piece of (human-usable) **information** can be represented as a string of **symbols**. To see why this hypothesis is useful, note these examples:

- `91772.3e-8`
- `Stay behind the yellow line!`
- ᐊᐃᖏᓚᐃ ᓄᓇᕐᔪᐊᖅ
- `/Pecs/Los Angeles/Berlin/Madrid//1 2/3 0/2 2/2 0/3 2///`

- `(* (+ 4 3) (-9 7))`
- `int average(int x, int y) {return (x + y) / 2;}`
- `<dog id="30024"><name>Lisichka</name><age>13</age></dog>`
- `{"type": "dog", "id": "30024", "name": "Lisichka", "age": 13}`
- `∀α β. α⊥β ⇔ (α•β = 0)`
- `1. f3 e5 2. g4 ♕h4++`

Symbols are chosen from some finite **alphabet**, such as $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ for the language of natural numbers, or $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \mathrm{e}, \mathrm{E}, +, -\}$ for the language of floating-point numbers. What is a language? A **language** (in Formal Language Theory) is a set of strings over an alphabet; that is, some strings are in-the-language and some are not. For example, in the language-of-floating-point-numbers, `3.55E+21` is in the language, but `+-9-E...2` is not. Beyond formal language theory, we consider a language to also contain an assignment of "meaning" to each string in the language.

Here's an obvious, but important, point worth stating: strings in a language represent not only numbers and text and complex data structures, but also abstractions of computations known as **functions**, which produce output information from input information. Examples:

- `double`, which takes a number and returns twice that number
- `plus`, which takes a pair of numbers and returns their sum
- `nextMove`, which takes a game state and returns the next (best?) move that should be made
- `upperCase`, which takes a piece of text and returns a string like it in every way *except* all letters are in upper case
- `sorted`, which takes a list of things and returns a new list containing the same values in sorted order
- `even`, which takes a number and returns true or false depending on whether the number is even
- `twice`, which takes a function and a value and applies the function to the result of applying the function to the value
- `javac`, which takes a string in the Java programming language and returns an executable class file
- `java`, which takes a class file and returns (the effect of) its execution
- `python`, which takes a string in the Python programming language and returns (the effect of) its execution

Functions are **applied** to **arguments** to produce their results. (The application is sometimes called a **invocation** or a **call**.) For example:

- `double(3.5)` $\Longrightarrow$ `7`
- `plus(4, -3)` $\Longrightarrow$ `1`
- `upperCase("Please don't shout!")` $\Longrightarrow$ `"PLEASE DON'T SHOUT!"`
- `sorted([4, 7, 1])` $\Longrightarrow$ `[1, 4, 7]`
- `even(797)` $\Longrightarrow$ `false`
- `twice(double, 5)` $\Longrightarrow$ `double(double(5))` $\Longrightarrow$ `double(10)` $\Longrightarrow$ `20`
- `python("print(8 * 5 - 2 / 1)")` $\Longrightarrow$ `{stdout: "38"}`

Functions can be expressed in so many ways. Alonzo Church invented the **Lambda Calculus** to show that function definition and application could serve as a foundation for mathematics. Here are example functions:

- `λx. not (equal (mod x 2) 0)`
- `λp. λn. λr. λt. times p (power (plus 1 (divide r n)) (times n t))`

Abbreviations are okay:

- $\lambda x\,.\,x \bmod 2 \neq 0$
- $\lambda(p, n, r, t)\,.\,p \times (1 + \frac{r}{n})^{nt}$

There are other notations, too:

- `(LAMBDA (x) (NOT (EQUAL (MOD x 2) 0)))` (Lisp)
- `lambda x: x % 2 != 0` (Python)
- `x => x % 2 != 0` (JavaScript)
- `(x) -> x % 2 != 0` (Java)
- `->(x) {x % 2 != 0}` (Ruby)
- `#(not= (mod % 2) 0)` (Clojure)
- `{$0 % 2 != 0}` (Swift)
- `|x| { x % 2 != 0 }` (Rust)

In Language Theory, we study how to **express** these functions, as well as the data upon which they operate, and the modules, objects, and control structures which house these functions. We study how **syntax definitions** can be written to define which strings belong to the language, and how **semantic definitions** can give meaning to these strings. We even study how these languages are used by human beings, something we call **pragmatics**. But how can we actually **compute** (or "carry out", or "evaluate") these functions? For that we turn to automata theory.

Automata Theory

In **Automata Theory**, we build and study formal models of computational devices (i.e., computers). We can start to develop a theory by thinking about how it is we expect our functions to be formally defined, and in some sense, how they "work".

Early Computing Machines

Of course, people have entertained the idea of building calculating machines for centuries. Notable examples include:

- Abacus
- Antikythera
- Astrolabe
- Pascal's adder
- Babbage's Analytical Engine
- Jacquard's Loom
- and more

These were all amazing feats of engineering and human ingenuity in their own right, but it wasn't until Turing that we finally had a solid understanding of what computation really is.

> *"Babbage never foresaw the terrible consequences of his invention—a machine that would autocorrect his name to 'cabbage' every single time." —Philomena Cunk*

Turing Machines

When Turing was in his early 20s, he set out to prove Hilbert wrong by solving the Entscheidungsproblem in the negative. To do so, he needed to define, formally, what was meant by computation, so he created a simple machine model, or **automaton**, to capture the essence of computation (or algorithm, or effective procedure). He called machines derived from this model **a-machines**, but we know them today as **Turing Machines**. Turing Machines are by no means the only kind of automaton that can carry out computations, but they are special, for a variety of reasons.



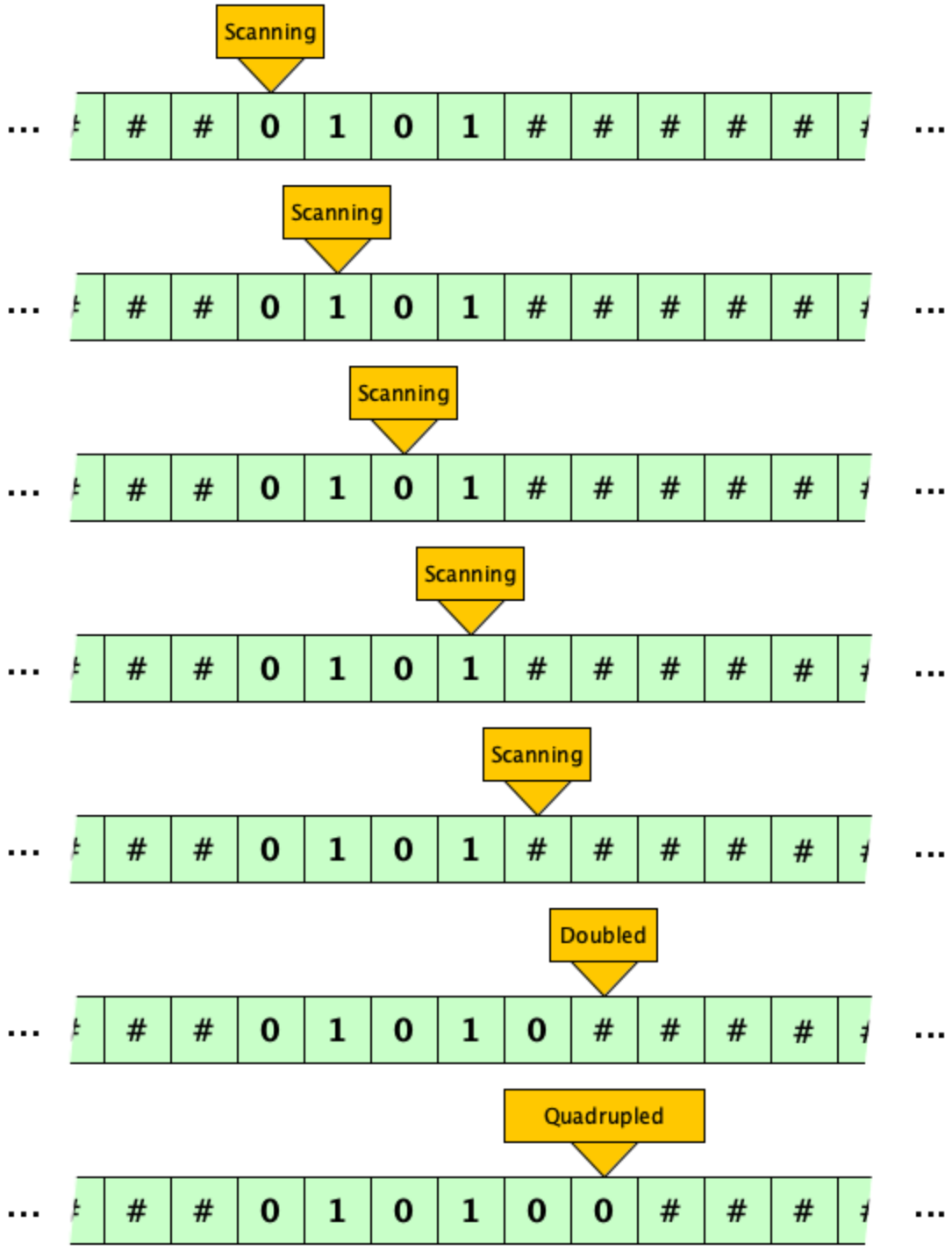
Turing set out to formalize what (human) computers did: they followed a set of instructions in which they examined some portion of their paper worksheet, wrote something down or erased something, then moved somewhere else or finished with their result. The actual instructions were based on the human's **state of mind** and what was already on the paper. He realized the contents of a two-dimensional sheet of paper could be just as easily represented as a one-dimensional sequence of symbols. So he then reduced the idea to a machine operating on an unbounded sequential paper **tape** with a read-write head initially pointing to the leftmost symbol of the input. At each step, the machine reads the current symbol, and depending on its current state, possibly rewrites it with a different symbol, moves one square to the left or right, and possibly enters a new state.

> ***Turing's Story***
>
> *History is important in understanding just about anything. The story of what motivated Turing to work on the foundations of computing and how he arrived at these fascinating machines, and his brilliant insights into the nature of computation are told very well in the book Turing's Vision: The Birth of Computer Science*
>
> *That book is well worth a read.*

Let's build a Turing Machine to multiply a binary number by four. How would we do this? Well, we just append two zeros to the end of the input! But how can we describe these steps more rigorously? What might the machine's instructions look like? We might say:

- Begin in the `Scanning` state, where our job is to look for the end of the input. In this state, we keep moving right until we encounter our first blank, which we replace with a 0 then move right, entering the `Doubled` state.
- In the `Doubled` state, we replace the blank with our second zero, entering the `Quadrupled` state.

Here is how our machine operates with the number five (binary 0101) initially on the tape (And oh, following convention, we use `#` for blank squares, just because it is easier to see!):

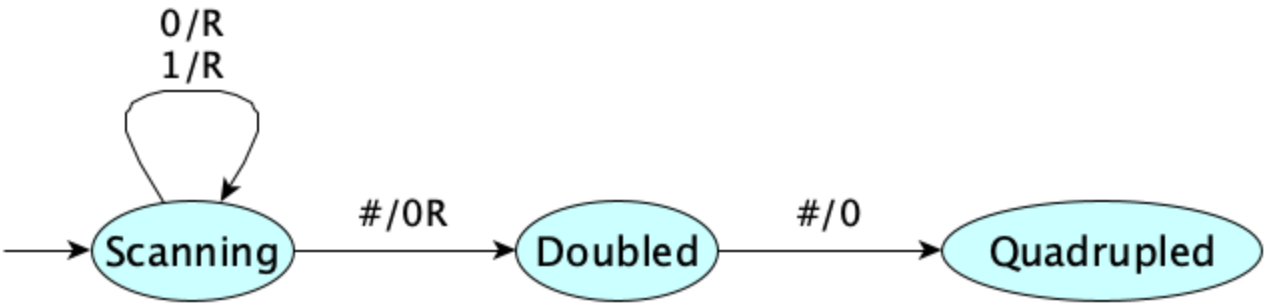


Nice! 5 × 4 is indeed 20 (binary 010100). Now let's capture the the multiply-by-four machine in all its glorious generality:

When in State	Looking At	Perform these actions	And go to State
Scanning	**0**	Move right	Scanning
Scanning	**1**	Move right	Scanning
Scanning	#	Print 0, Move right	Doubled
Doubled	#	Print 0	Quadrupled

This says: starting in the `Scanning` state, while you see a 0 or 1, leave it alone and move right, but if you see a blank, replace with a 0, move right, and go to the `Doubled` state. In this state, if you see a blank, replace with a 0 and go to the `Quadrupled` state. There is no place to go from here, so we're done.

We can give a pictorial representation of the TM program as a **state diagram**:



CLASSWORK

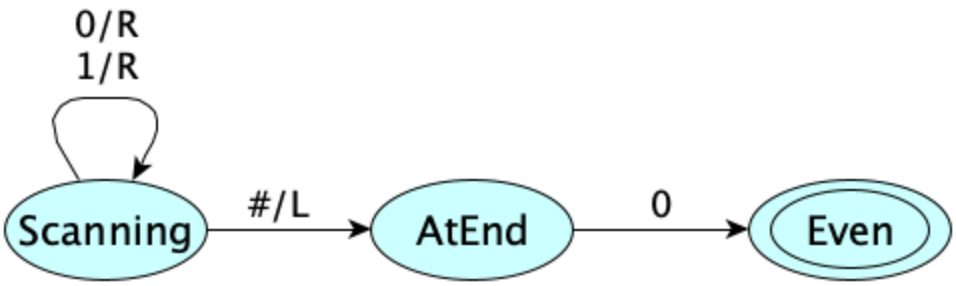
> Create a Turing Machine that **floor-divides** its input by four. This will be a little harder, as you will have to sometimes move left, and handle the case in which the input is very small and you will need to make sure an answer of 0 is properly written out.

> ***Exercise**: Give Turing machines that (a) increment a binary number by 1, (b) compute the two's complement of a binary number, and (c) compute the value of a binary number modulo 2.*

TMs for Decision Problems

If a Turing Machine is created to answer a yes-no question (a **decision problem**), such as "Is this number even?" or "Is this string a palindrome?" it theoretically could erase the whole tape and write a YES or NO symbol as soon as it figures out the answer. But there is a shortcut we almost always take: we can mark certain states as **accept states**. In such a TM, we don't care what remains on the tape—we only care whether the state we end up in when there are no moves to make is an accept state or not.

Example Time! Let's give a TM to determine whether a binary number is even:



Hey, there is a connection to language theory here! The TM just given can both answer the question "Is this number even?" but it also **recognizes the language** $\{w \in \{0,1\} \mid w \text{ is a even binary integer}\}$. That's right, answering yes-no questions is isomorphic to determining membership in a language.

***Exercise**: Reflect on this. Woah, right?*

***Exercise**: Give Turing machines that determine whether (a) a binary integer is odd, (b) a signed binary integer is negative, and (c) a string over $\{a, b, c\}$ is a palindrome.*

Universality

In principle, we can build Turing Machines for computing all kinds of functions, such as capitalizing text, finding words in text, computing interest payments, finding documents on the World Wide Web, etc. But functions themselves can be represented in symbols, whether in mathematical notation, plain text Python, or any formalism of your choice. In fact, the Turing machine we saw to multiply by four could be represented as:

```
S00RS;S11RS;S#0RD;D#0NQ;
```

Given this, perhaps we can build a machine (call it U), that when you give it ANY function f (as text), and some data x, then U will produce $f(x)$? That is:

$$U(f, x) = f(x)$$

As it turns out, for a certain class of functions, you *can* construct such a machine. You might take that for granted today, but it was not shown to be true until the 20th century! Really! (As far as anyone knows, that is.) This profound result is due to, you guessed it, Alan Turing, and it pretty much changed the world. This U is called the **Universal Turing Machine** in his honor. Today we build variations of the original U. We call them **computers**.

Stop.

Think about this.

Woah.

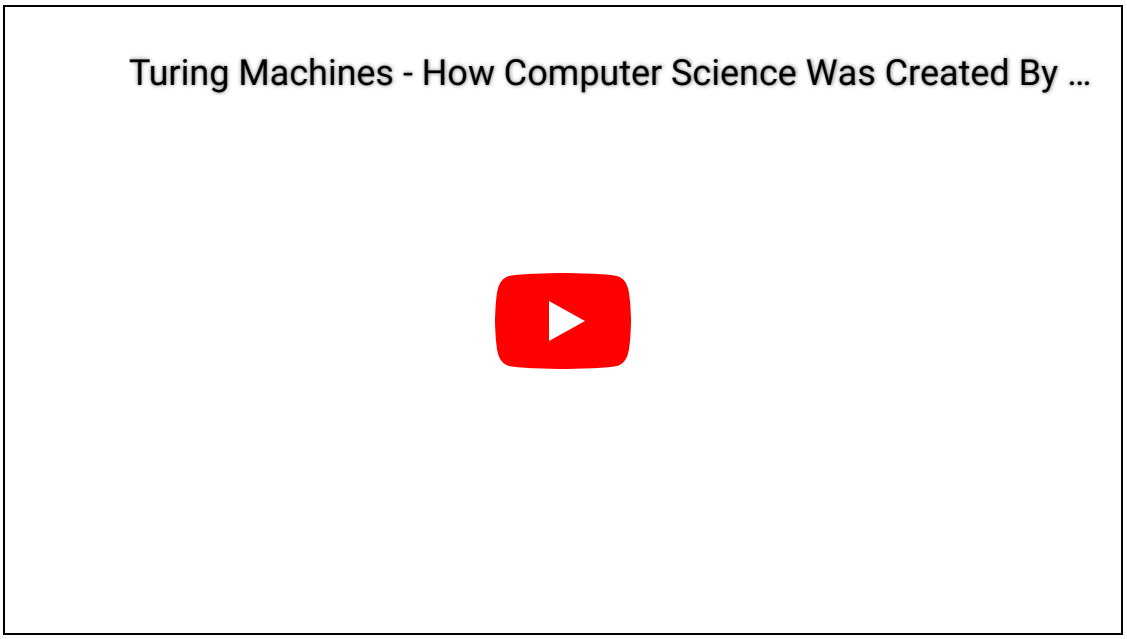
Just woah.

This may the most profound idea anyone has ever had about computing.

Are you starting to see why Alan Turing is revered as the founder of computer science?

Jade explains it better

Here's Jade Tan-Holmes explaining all about Turing Machines and Universality. She explains what motivated Turing to come up with the idea for formalizing computation, how he came up with the model he did, and the impact of universality. This video is so spot on, you need to watch it. Maybe watch it twice:



Other Kinds of Automata

Turing Machines are not the only model of what we think of as computation (though they might be the most famous). Other models that have been created include:

- Register Machines
- The Lambda Calculus

- Post Systems
- Partial Recursive Functions
- Analytic Grammars
- Queue Automata
- Stack Automata
- String Rewriting Systems
- Cellular Automata

Some have equivalent "computability and recognition power" as Turing Machines, while others do not. What do we mean by "computability and recognition power"? That's the domain of our next theory, computability theory.

Computability Theory

Now for our next philosophical question. **Can *every* function be represented as a string of symbols that we can process with a machine? In other words, is there a program for every function?**

It turns out that the answer to this is no.

Here's why. Programs are just strings, so we can list them in increasing alphabetic order. Programs can be thought of as representing functions in $\mathbb{N} \to \mathbb{N}$ (since all data can be encoded in binary). Call the enumeration $f_0, f_1, f_2, \ldots$. But $\lambda n.\ f_n(n) + 1$ isn't in the list! Bottom line:

There exist functions that cannot be computed by a computing machine.

Hey, *this is philosophically interesting*.

***Exercise**: Discuss why the limits to computation might have something to do the possibility of finding a "Theory of Everything". (Hint: See what Stephen Hawking has to say about this.)*

CLASSWORK

That explanation of why uncomputable functions exist was given rather nonchalantly. Did it make sense? It might not have right away because it is an instance of a very powerful proof technique called **diagonalization** which was at first somewhat controversial. Carry out this proof by sketching the hypothetical list of all functions as a matrix: each row is a function and each column is the argument, and each cell is the result of applying the function (row) to the

argument (column). Look at how $\lambda n.\ f_n(n) + 1$ is generated. Do you see why the technique is called diagonalization? Do you see why it works? 🤯

So which functions can be computed? People have, over the years, created many models of computation—the Lambda Calculus, Post Production Systems, Phrase Structure Grammars, Partial Recursive Functions, Random Access Machines, and more. But get this: all have turned out to be **equal in power, in terms of what they can compute, to the Turing machine**! Even enhancing the basic Turing Machine design with **non-determinism** or even **multiple tapes** won't increase its computing power. These facts suggest the class of computable functions is fundamental. The **Church-Turing Thesis** claims that what we intuitively understand as the "computable functions" are precisely those for which a Turing Machine can be given, i.e.,

A function f is **computable** if and only if a there exists a Turing Machine that outputs $f(x)$ for any input x.

***Exercise**: Research why this is called a thesis and not a law or theorem.*

Remember that function U we saw before? The Universal Turing Machine? It is "universal" because it can simulate any Turing Machine. If the Church-Turing Thesis is true, it's *really* universal: it can compute any computable function.

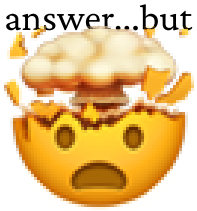
***Exercise**: Discuss why this is profound.*

Computability Theory is concerned with figuring out which functions are and are not computable.

> ***Turing Complete***
>
> *Because Turing Machines capture that which is computable, any computational model that can simulate a Turing Machine is called* **Turing Complete**. *Being Turing-Complete is a badge of honor, as it means your model can compute anything that is computable.*
>
> *Read more at Wikipedia.*

Fun fact: One of the more interesting things to come out of computability theory is that there are some languages that Turing Machines can recognize, but not decide. (A TM recognizes a language L if it enters an accept state and halts whenever given a string in L. It **decides** a language L if it both accepts L *and* for all strings not in L, the machine will always stop in a non-accept state.) So what

does it mean that some languages are recognizable but not decidable? It means there are some languages—and therefore some decision problems— for which we can definitely compute a YES answer...but when the answer is NO we might...never...know.



How about an example? We'll use JavaScript rather than TMs to simplify the explanation. Let's write a function called `halts` that will determine if a function halts on a given input. So we would like:

```
halts(Math.sqrt, 100)                 → true
halts(x => {while (x) {}}, true)      → false
```

The idea is simple. Just analyze the input and see if it will halt. But it turns out to be logically impossible to write this function. Why? Assume you can write it. Then consider:

```js
function fox(f) {
  if halts(f, f) while (true) {}
}
```

Now consider the expression `halts(fox, fox)`. Woah. We have an antinomy. The assumption you could write `halts` leads to a contradiction, so you can not write this function to work for all inputs. Interestingly, you can write:

```js
function halts(f, x) {
  f(x);
  return true;
}
```

So if `f(x)` halts, you'll eventually know it, but if it doesn't, we may never know.

Complexity Theory

Computability theory gives us theoretical limits on what can and cannot be computed, but doesn't tell us what is **practically** computable. (A problem requiring 2^{100} years to complete is not worth

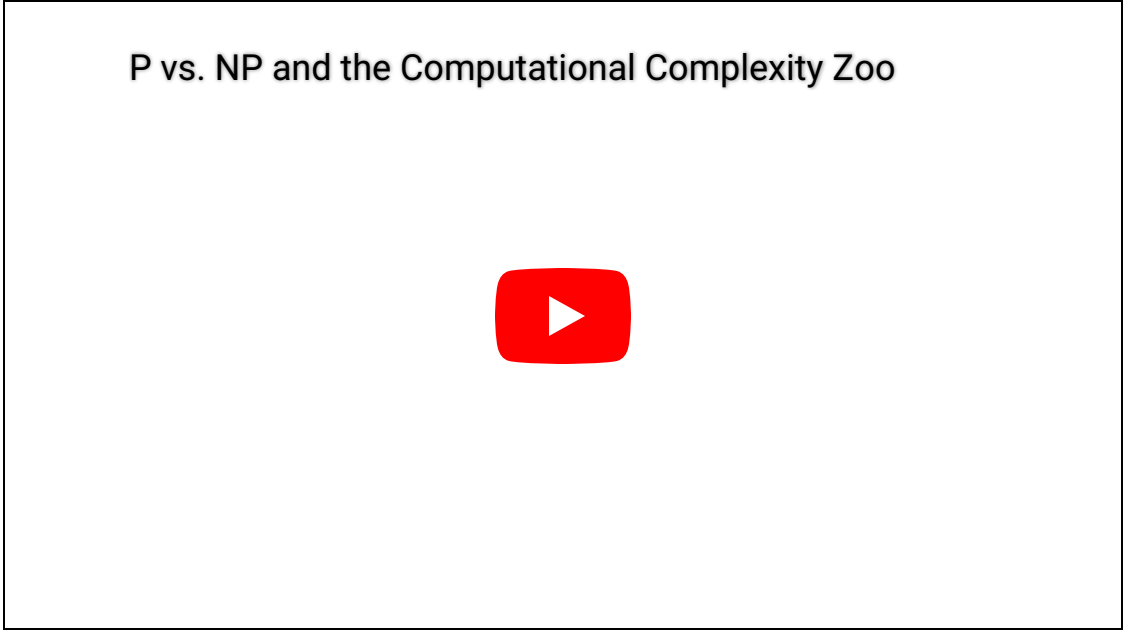
running.) The question “What resources are required to perform certain computations?” or in other words “How much time and memory is needed to solve this problem?” give rise to **Complexity Theory**.

Complexity Theory gives us results such as “This function requires _____ to compute”:

- only a single step
- only a fixed number of steps
- logarithmic time
- linear time
- quadratic time
- polynomial time
- exponential time
- no more than a single cell of scratch memory
- scratch memory in the form of a stack (which is very efficient)
- logarithmic space
- linear space
- polynomial space
- exponential space

It gets (massively) more interesting, because the theory deals with time and space performance on non-deterministic machines, and parallel machines, and interactive machines, and quantum computers, too. It even considers **probabilistic algorithms**. Interested? Check out the amazing Complexity Zoo.

Complexity Theory has philosophical as well as practical use. Make sure you are familiar with one of the most famous open questions in all of complexity theory, and perhaps even all of computer science, namely: Does $P = NP$? (Solve it and you will win a million dollars!)



Oh, wait! Here's Jade again with another great explainer:



There is so, so, much information out there about the P vs. NP question as it really seems to captivate the public. Not unlike Gödel's Incompleteness Theorems.

***Exercise**: Scott Aaronson wrote a blog post in 2006 (worth a read) in which he waxed poetic about a world in which P=NP. He later disavowed some of his more colorful claims, writing:*

> *I, Scott Aaronson, hereby formally disown my statement about how "everyone who could appreciate a symphony would be Mozart" if P=NP.*

In 2014, he wrote a new post on the scientific case for P≠NP. Read the later post. What do you think?

Working With These Theories

We've only briefly introduced the major concerns of four computer science theories without any real depth (or breadth). We'll have a chance to see each theory in more detail later, covering aspects such as:

- Formal, mathematical, definitions of languages (via grammars) and automata
- Restricted grammars and automata that capture specific kinds of computation
- Language hierarchies capturing both computability and complexity concerns
- Models of computation other than the Turing Machine
- More examples of non-computable functions (beyond the "Halting Problem" we saw above)
- How to construct a Universal Turing Machine
- Examples of languages in many of the common complexity classes
- How to determine the complexity of a problem
- How to create grammars for realistic programming languages
- Why non-trivial programming languages require both a static and dynamic semantics
- Extending these ideas into the much larger Programming Language Theory (PLT)
- How to run a program given its source code (interpret)
- How to translate one programming language into another (compile)

Theories and Creativity, Again

In much the same way that Bennett pointed out that music theory can expand your abilities in music composition, and Feynman spoke of theoretical knowledge in science only expanding and enhancing the feelings of awe and beauty when contemplating nature, might computer science theory enhance one's imagination for creating new worlds that Booch talks about here?



I am a computer scientist; I approach the cosmos though a very different path: I take the simplest of principles - information theory, logic, design - and from them create new worlds that are bound only by

my imagination.

— Grady Booch (@Grady_Booch) February 12, 2020

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What exactly is a **theory**, in the context of art and science? HIDE ANSWER

An organized body of knowledge with explanatory and predictive powers.

Why are theories important? HIDE ANSWER

They provide us with a vocabulary to communicate ideas, test hypotheses, and generate new knowledge.

What are the four major computation theories and what are they concerned with? HIDE ANSWER

1. Language Theory: expressing computations
2. Automata Theory: carrying out computations
3. Computability Theory: What can and cannot be computed?
4. Complexity Theory: Resources required for certain computations

What is a language, in formal language theory? HIDE ANSWER

A set of strings over some finite alphabet.

What is the lambda calculus seemingly most concerned with representing and manipulating? HIDE ANSWER

Functions

Who came up with the idea of formalizing computing via the Lambda Calculus? HIDE ANSWER

Alonzo Church

Who came up with the idea of formalizing computing via the Turing Machine? HIDE ANSWER

Alan Turing

What was Turing trying to do when he realized he would have to do no less than formally define the very notion of an algorithm (or computational process)? HIDE ANSWER

He was trying to show Hilbert's Entscheidungsproblem did not have a solution

On what did Turing more-or-less base his fundamental idea of how a formal (abstract mechanical) computing device should work? HIDE ANSWER

(Human) computers, calculating on paper by moving through mental states looking at symbols, adding new symbols, and erasing existing ones.

What are the main components of a Turing Machine? HIDE ANSWER

1. A control unit made up of states and a transition relation, and
2. An infinite one-dimensional tape with cells that can each hold a single symbol or be blank.

Usually, a Turing machine just rewrites its input into an output. But sometimes, all we want to compute is the answer to a YES-NO question. In this case, how does the Turing Machine announce its answer? HIDE ANSWER

It says YES by entering an ACCEPT state with no outgoing transitions for the current symbol, and answers NO by getting into a non-ACCEPT state with no outgoing transition for the current symbol. It is possible for the machine to

loop forever on a given input, in which case it is said to give no answer.

Answering a YES-NO question on a Turing Machine is isomorphic to _____ a language.

HIDE ANSWER

Recognizing

Why is Turing's discovery/invention of the Universal Turing Machine so profound?

HIDE ANSWER

It means that truly general-purpose, programmable computing machines exist (i.e., we don't need specialized machines for all computations).

How can we prove there are functions that cannot be computed? HIDE ANSWER

By diagonalizing over an assumed enumeration of all functions.

What does the Church-Turing Thesis say? HIDE ANSWER

That the Turing Machine lines up exactly with our notion of what is intuitively computable.

What is the difference between *deciding* and *recognizing*? HIDE ANSWER

A TM decides a language if it always halts with a YES or NO answer. A TM recognizes a language if it always correctly answers YES for strings in the language, but it may or may not halt with a NO for non-members.

What are the language classes P and NP? HIDE ANSWER

P is the set of languages decidable in polynomial time on a deterministic TM; NP is the set of languages decidable in polynomial time on a nondeterministic TM.

Does P = NP?

Did Scott Aaronson really write “everyone who could appreciate a symphony would be Mozart”? How does he feel about writing that these days? HIDE ANSWER

He did indeed write that but he regrets writing it now.

What are some variations we can make to Turing Machines that possibly affect complexity (if not computability)? HIDE ANSWER

Randomized machines, probabilistic machines, quantum machines.

How does Grady Booch distinguish the work in material domains (physics, chemistry, biology, etc.) from that of computer science? HIDE ANSWER

Scientists in material domains observe the cosmos and reduce it to simple principles; computer scientists start with simple principles and from them create new worlds bound only by the imagination.

Summary

We’ve covered:

- ✅ What a theory is
- ✅ Why study theory?
- ✅ Theories within computer science
- ✅ What Language Theory is
- ✅ What Automata Theory is
- ✅ What Computability Theory is
- ✅ What Complexity Theory is

✅ Where we can go from here

Computability Theory

Having theories about what computations are and how they are expressed is cool, but how about some insights into what can and cannot be computed?

CONTENTS

History • Models of Computation • Limits of Computation? • A First Non-Decidable Problem • Recognizability • Beyond Recognizable • Reductions • Rice's Theorem • The Chomsky Hierarchy • Oracles • Connections to Complexity Theory • More Resources • Summary

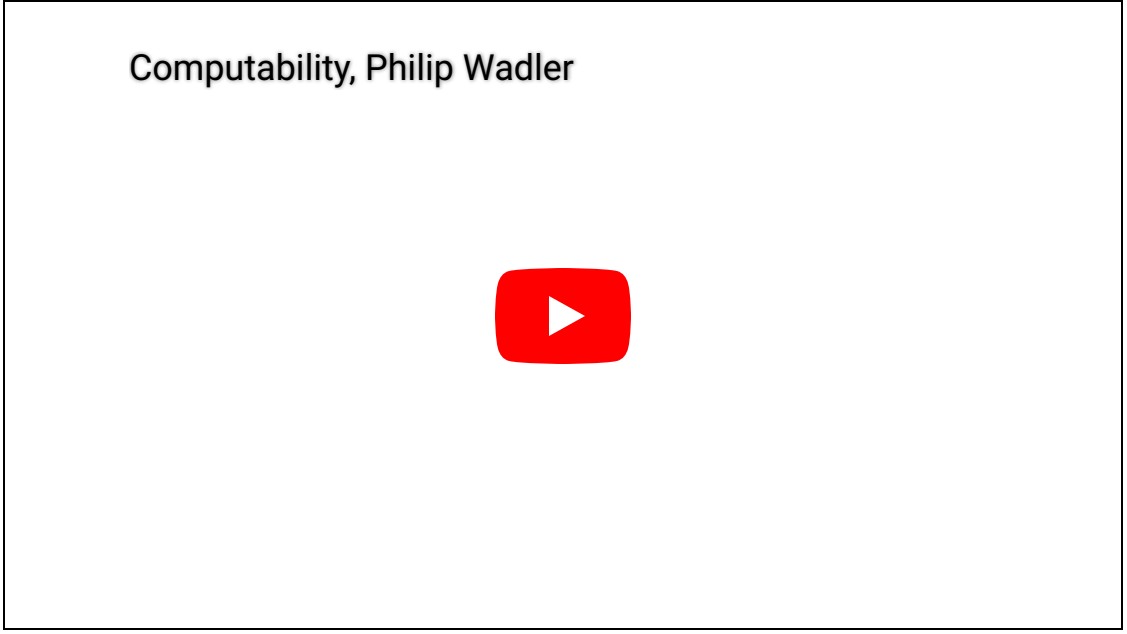
History

David Hilbert was good at formal logic. He wondered about many things.

He challenged mathematicians of his day with many problems.

Thinking there might be a way to algorithmically determine the truth or falsity of any statement in an axiomatic system, he posed the Entscheidungsproblem.

Philip Wadler tells the story:



Computation has limits. Mathematicians and programmers will never go out of business. Computer Science will always be an interesting field. We can dig deeper into this field of computation, and in particular, study what it means to be computable.

Models of Computation

We learned form Wadler that all three of these:

- Church's λ-calculus
- Gödel's μ-recursive functions
- Turing's Turing Machines

were equivalent. It turns out that a bunch of other efforts to formalize computation were also equivalent:

- Chomsky's Phrase-Structure Grammars
- String Rewriting Systems (also known as Semi-Thue Systems)
- Post's Post Production Systems
- All sorts of crazy extensions to Turing Machines (except magic oracles)

Because all of these formalisms are equivalent, and no one has found something better (other than magic oracles), most folks hypothesize that **we really have formally captured our intuitive notion of computability** with these things. The **Church-Turing Thesis** states that what we intuitively understand by "computation" is captured by the Turing Machine.

Limits of Computation?

In retrospect, one wonders *why* Hilbert suspected that every mathematical statement (i.e., a statement of a "formal axiomatic systems capable of representing arithmetic and beyond") was decidable! And thus, whether all true statements had proofs, or why there should exist a finite algorithm that could compute all functions in $\mathbb{N} \to \mathbb{N}$ (or even in $\mathbb{N} \to \mathbb{B}$).

For example, a simple diagonalization proof tells you there are non-computable functions in $\mathbb{N} \to \mathbb{N}$:

> **Proof**: Programs are finite strings of symbols so there are a countably infinite number of them. If all functions were computable, then there would need to be a program for each one, so there would need to be a countably infinite (enumerable) number of functions. If we *could* enumerate every function from $\mathbb{N} \to \mathbb{N}$ as $f_0, f_1, f_2, \ldots$, then the function f such that $f(n) = f_n(n) + 1$. would not be on the list. So the list is incomplete.

CLASSWORK

We're going to write out this **diagonalization** on the board.

***Exercise**: Prove that the number of functions from $\mathbb{N} \to \mathbb{B}$ is also uncountable.*

Okay nice, so there are more functions than programs. But are there any realistic functions we can specify that we cannot decide?

A First Non-Decidable Problem

Do there exist problems for which all instances have a YES-NO answer but no computer, no matter how powerful, nor any algorithm, in any formalism whatsoever, can compute the answer?

Of course, as we've shown above, a simple counting algorithm tells us that yes, they must exist! But are there any that we can describe and actually understand? Yes, and here's the classic first example. It is known as **The Halting Problem**:

> Write a function called `halts`, which accepts a function `f` and an input `x`, and returns `True` if `f(x)` halts, and `False` if `f(x)` enters an infinite loop when run.

Sounds simple enough. Let's **assume** we can write it:

```
def halts(f, x):
    # deciding the issue and return True or False
```

But here's the problem. Let's consider this, perfectly normal function:

```
def fox(f):
    if halts(f, f):
        while True: pass
```

The expression `fox(fox)` leads to an inconsistency: Running `fox(fox)` does this: "if `halts(fox, fox)` then go into an infinite loop, otherwise halt. Ugh. Wait. OMG. This cannot be. So our **assumption that `halts` could be written must be incorrect.** We cannot write `halts` in any programming language. The halting problem is undecidable.

> ***Python?***
>
> *It's way more fun to use Python in these proofs, but when we do, keep in mind these are isomorphic to language decision problems. In the Halting Problem case, we can re-cast this whole "problem" into one of deciding a particular language, namely:*
>
> $$\mathcal{H} = \{\langle M \rangle w \mid M \text{ halts on } w\}$$
>
> *where $\langle M \rangle$ is the string encoding of machine M.*

Recognizability

But guess what: Even though you can't **decide** whether a program will halt, you *can* recognize whether a program will halt. Just simulate it. So the language of halting programs is **recognizable but not decidable**.

If you can recognize a language, you *can* enumerate all its utterances. For concreteness, let's see how this is done with the Halting Problem. Since all programs are strings we can enumerate a list of all programs $p_0, p_1, p_2, p_3, \ldots$. We can do this:

- At time t_1, run one step of p_0
- At time t_2, run two steps of p_0 and one step of p_1
- At time t_3, run three steps of p_0, two steps of p_1, and one step of p_2
- ...

...and so on. Whenever a p_i halts, add it to your "output list" of halting programs.

This process enumerates all and only the halting programs.

We have the following fact:

> The set of all languages that are Turing-Recognizable is known as RE, namely, the set of **Recursively Enumerable Languages**.

and also:

> The set of all languages that are Turing-Decidable is known as R, namely, the set of **Recursive Languages**.

Beyond Recognizable

But now, what about the language of all strings that encode Turing Machines that never halt (sometimes called $\bar{H}$? That language isn't even recognizable. This will need a proof, but we'll get there soon.

So, how do we know a language isn't recognizable?

We can find a *first non-recognizable language* using diagonalization. The language is called $\mathcal{X}$ and is defined by:

$$\mathcal{X} = \{\langle M \rangle \mid M \text{ does not recognize } \langle M \rangle\}$$

(Informally, it is the set of all machines that don't recognize themselves.) To show this language is not recognizable by any Turing Machine let's make a list of all the Turing Machines there can possibly exist. There are a countably infinite number of these. We will call them $M_0, M_1, M_2, \ldots$. For each machine we will write out whether it recognizes the encodings of all of the machines:

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...

M_0	T	F	F	T	...
M_1	T	T	T	T	...
M_2	F	F	F	T	...
M_3	T	T	F	T	...
⋮	⋮	⋮	⋮	⋮	⋱

The T's in each row comprise the language recognized by M_i. The language $\mathcal{X}$ is the flipped diagonal of this table, which cannot appear in this table. No matter how we try to enumerate all the Turing Machines, there will never, EVER, be a row which makes $\mathcal{X}$. So $\mathcal{X}$ cannot be recognized by a Turing Machine.

Reductions

So the Halting Problem is undecidable. But how do we prove other problems undecidable? One way is to prove: "If problem P were decidable, then the Halting Problem would also be decidable."

This is done by **reducing** the Halting Problem to problem P.

In general, a **reduction** of problem A to problem B means we use an algorithm for B to solve A. You've probably done that a lot in the past, right? You've written "utility functions" and then called them to do a larger task.

Let's to this for the problem of determining whether $L(M) = \varnothing$. In formal notation, we want to know if we can decide:

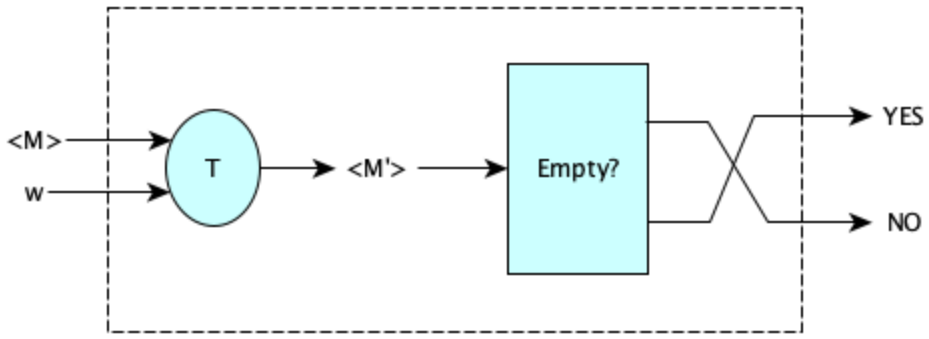
$$\mathcal{E} = \{\langle M \rangle \mid L(M) = \varnothing\}$$

This is the empty language problem. We can reduce the halting problem to it. Let's do it. Suppose $\mathcal{E}$ were decidable. This would mean we would have a machine E that when given the description of a machine M as input, would always halt with YES when $L(M) = \varnothing$ and halt with NO otherwise.

So let's make a machine called H that (1) takes two inputs, $\langle M \rangle$, the description of a Turing Machine, and w a string, and produces the description of a machine that erases its input, writes w on it, and simulates M, then (2) passes that new machine to E, then (3) swaps the outputs of E. Machine H

decides the halting problem. But this cannot be. The only place we could have gone wrong was our assumption that a decider for E exists. So $\mathcal{E}$ is undecidable.

A diagram might help:



Here $L(M') = \varnothing$ iff M does not halt on w, so the dashed box is a halting problem decider, which should not exist. So the Empty? machine cannot be a decider.

See Wikipedia's article on reducing problems and Wikipedia's list of undecidable problems.

Rice's Theorem

Here is something amazing.

Let's say you were given a general YES-NO problem about the run-time behavior of programs (e.g. does the program halt? or does this program ever output more than three symbols?). **Rice's Theorem** says that the problem is undecidable unless the property holds for all programs or holds for none.

More technically: All nontrivial semantic properties of programs are undecidable.

Here semantic means run-time and trivial means holds for all or holds for none.

More at Wikipedia.

> ***Exercise**: Research whether Rice's Theorem applies to languages as well. For example, the question "Is this language context-free?" is a YES-NO question. Clearly some languages are context-free and others are not. So the question is nontrivial. Does Rice's Theorem apply to this question?*

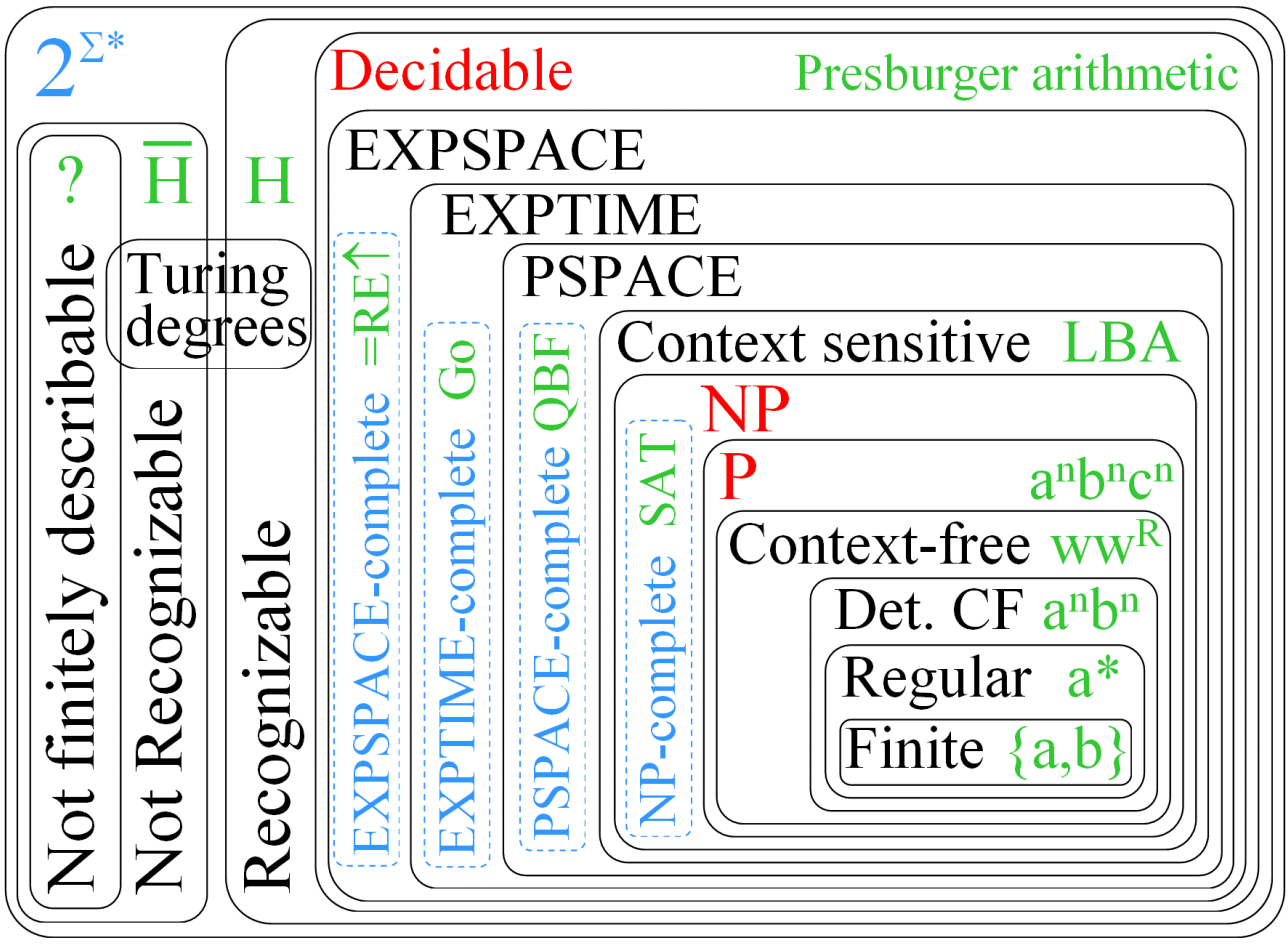
The Chomsky Hierarchy

The “limits” of computing models can define families of languages. For example:

The languages that can be recognized by	**are the ___ languages**	**which Chomsky called**
Turing Machines	Recognizable, a.k.a. Recursively Enumerable (RE)	Type 0
Turing Machines that always halt	Decidable, a.k.a. Recursive (R)	
LBA	Type-1 (CS)	Type 1
NPDA	Context-Free (CF)	Type 2
DPDA	Deterministic Context-Free (LR)	
FA	Regular (REG)	Type 3
FA with no cycles	Finite (FIN)	

ALL OF THESE LANGUAGE FAMILIES ARE PROPER SUBSETS OF THE FAMILY IN THE ROW ABOVE IT.

Gabe Robins has a beautiful visualization (which mixes in a few classes from complexity theory):



Oracles

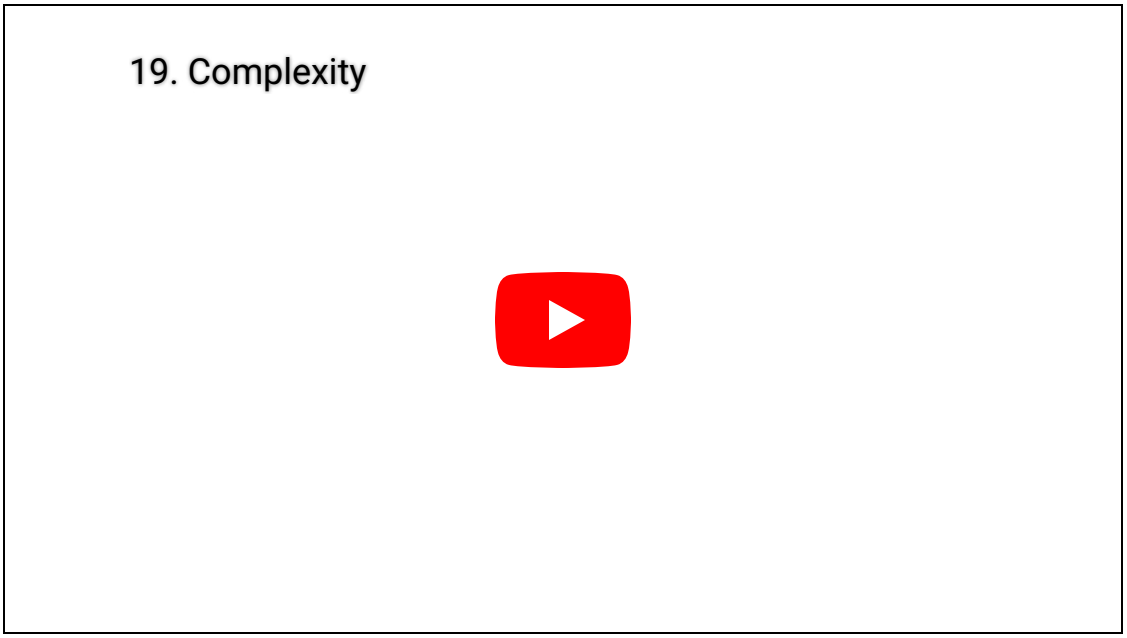
Loosely speaking, an **oracle** is something you can add to a machine, that, at any time during a computation, you can simple "ask the oracle" a question whose answer might not even be computable. It just magically answers the question and you go about your business.

Read about oracles at Wikipedia.

Connections to Complexity Theory

Computability Theory deals with what can and cannot be computed on a particular computing model. It does not make any claims on the number of steps required, or the amount of space required, to do the computation. That is what complexity theory is for.

But you know, maybe we *shouldn't* try to separate computability from complexity. These are convenient labels, but they are very inter-related. In fact, Robins's diagram intentionally places complexity classes in with the standard computability classes. Many schools offer a course called *Computability and Complexity*. Erik Demaine at MIT gives a brief overview of the limits of computation within his complexity theory lecture in his 6.006 class:



***Exercise**: Demaine expresses surprise that, and says it is depressing that, most problems are undecidable. Do you share that view? My personal view is one of wonder and amazement that we can actually know what the decidable problems are, given that virtually all problems are undecidable (in other words the "probability" that an arbitrary function is computable is zero). What is your take? Depression, or Awe? (Demaine does say we're "lucky" at least to know how to solve some problems, at least.)*

More Resources

You might also like:

- Gabe Robins's Theory Course Notes from University of Virginia
- Some nice notes from a Columbia course

When studying Robins's notes, focus on the set entitled "Formal languages, automata models, grammars, decidability, and Rice's theorem" (PDF here). There are several examples of problem reductions and of applications of Rice's Theorem.

Summary

We've covered:

- ✅ A history of computability theory
- ✅ Models of computation
- ✅ Limits of computation
- ✅ The halting problem
- ✅ The difference between recognizability and decidability
- ✅ The idea of problem reduction
- ✅ Rice's Theorem
- ✅ The Chomsky Hierarchy
- ✅ Oracles
- ✅ Connection to Complexity Theory